

Feasibility Pump with Objective Cutoff: A Computational Study

Matteo Fischetti*

September 2026

Abstract

A common way to make the Feasibility Pump objective-aware is to add a cutoff $c^\top x \leq U$ to the model and run the pump on it as a black box. Fischetti, Glover and Lodi (FGL) warned against exactly this: rounded points that are feasible for the original constraints but not for the cutoff are lost unless one looks at them, and they prescribed a recovery, a feasibility test or an LP with the integers fixed and no cutoff. Many implementations do not perform it.

We measure what the omission costs. On a from-scratch pump, two thirds of the feasible rounded points on covering problems fall above the cutoff, and the lost improving points pile up just above it, wherever it is placed. On MIPLIB 2017, a two-factor study, recovery by cutoff level, runs the pump as a standalone heuristic inside SCIP on the same instances, seeds and budget, every solution revalidated. Adding FGL's cutoff and recovery to the objective-weighted pump of SCIP beats that pump on 116 of the 176 instances of 424 where it already reaches a solution, loses on 9, and lowers the median final gap from 59% to 30%. Where the pump reaches nothing, no variant helps. Asked for a solution below a stated target, with the FGL pump written from scratch and no objective weighting, the cutoff in the model is a strong baseline, beaten only at a shallow target and only by the completion. The recommendation is the old one: keep the cutoff, and the recovery that goes with it.

Keywords Feasibility pump; Primal heuristics; Mixed-integer programming; Objective cutoff; MIPLIB 2017; SCIP

Mathematics Subject Classification 90C11; 90C59; 90C10

1 Introduction

The Feasibility Pump (FP) of Fischetti, Glover and Lodi [13], FGL in what follows, looks for a feasible solution of a mixed integer linear program $\min\{c^\top x : x \in P, x_j \in \mathbb{Z} (j \in I)\}$, $P = \{x : Ax \leq b, l \leq x \leq u\}$; let $B = \{j \in I : l_j = 0, u_j = 1\}$ be the set of binary variables, which we assume nonempty. The pump maintains two sequences of points that satisfy complementary halves of the problem. The LP iterates $\tilde{x}_0, \tilde{x}_1, \dots$ live in P and are in general fractional; the rounded points $\hat{x}_0, \hat{x}_1, \dots$ are integral on I by construction and in general violate $Ax \leq b$. Starting from the LP optimum $\tilde{x}_0$, each round rounds the current LP point to $\hat{x}_k$ and projects it back onto P ,

$$\tilde{x}_{k+1} = \arg \min\{\Delta(x, \hat{x}_k) : x \in P\}, \quad \Delta(x, \hat{x}) = \sum_{j \in B: \hat{x}_j=0} x_j + \sum_{j \in B: \hat{x}_j=1} (1 - x_j), \quad (1)$$

which is linear as written; the distance is taken on the binary variables, as in [13], and general integer variables are handled as in [3] (the instances of this paper are mostly binary). The

*Department of Information Engineering, University of Padova, Italy. matteo.fischetti@unipd.it

pump stops when $\tilde{x}_{k+1}$ is integral, and that point is a feasible solution; when the rounding repeats itself the pump perturbs $\hat{x}_k$ before projecting again. Every mechanism discussed below acts on the LP (1): it changes the polyhedron the LP iterate is projected onto and leaves the rounding alone. The pump has a large progeny [5, 6]: different projections [7, 2], better randomisation [12], and a weighted objective term in place of a pure distance [1, 21], which is what SCIP runs by default.

The cutoff, and the warning that came with it. The pump as stated is blind to the objective. Section 3.2 of [13] proposed the simplest remedy: once an incumbent of value z_{inc} exists, add to the LP an *objective cutoff*

$$c^\top x \leq U, \quad U = z_{\text{low}} + \lambda(z_{\text{inc}} - z_{\text{low}}), \quad \lambda \in (0, 1], \quad (2)$$

with z_{low} a lower bound on the optimum that the solver has at hand. Here $\lambda = 1$ puts the cutoff at the incumbent, which any solver imposes on its node LPs anyway, up to the improvement tolerance, and smaller λ pushes it towards the bound. This is also what a practitioner does when the pump is a black box: add the constraint to the model and run the pump on the restricted problem, hoping for a feasible solution of it. The same section warned against doing only that. Because of the cutoff, “it is often the case that the integer components of x [...] define a feasible point for the original system $Ax \geq b$, but not for the current one”; the remedy was to look at the rounded point every round and, on a mixed problem, to “solve the LP $\min\{c^\top x : Ax \geq b, x_j = \hat{x}_j \ \forall j \in I\}$ and compare the corresponding solution (if any exists) with the incumbent one”. (The quotation is in the notation of [13], where the system is written $Ax \geq b$ and I is the set of integer variables.) Cutoff and recovery were designed as two parts of one scheme.

Many implementations keep neither (Section 4 reads two of them). Later work replaced the cutoff by a weighted objective [1], mentioned it in passing [16], or dropped the objective altogether [8, 10]; and common implementations, the pump of SCIP [9, 22] among them, never test the rounded point at all (Section 4). What the black-box use of a cutoff costs, and what the recovery is worth, has not to our knowledge been measured. This paper measures both.

What we find. The main result is a restoration. The objective-weighted pump of [1], as SCIP runs it, with the cutoff at mid-gap and the completion of [13] added to it, beats that pump alone on 116 instances of 176, ties on 51 and loses on 9, and lowers the median final gap from 59% to 30%; against the black-box cutoff alone it wins 87–14, and against the cheapest sound alternative, the test of the rounded point without a cutoff, 114–10 (Section 6). The black-box cutoff already helps, 90–14 over the plain pump; the recovery is what makes it help more, and the completion pays where the cutoff is tight, on mixed problems, and nowhere else. Behind the result is a phenomenon. The rounded point $\hat{x}$ is often feasible for the original constraints, on covering problems almost always; when it is, it typically costs more than the LP iterate, which sits on the hyperplane $c^\top x = U$ when the cutoff bites, so it violates (2), lies outside the polyhedron the next LP projects onto, and a pump that does not look at it never sees it again. On a from-scratch pump a large share of the feasible rounded points is lost this way, two thirds on covering problems, and the lost improving points are not spread over the band between the cutoff and the incumbent but pile up just above the cutoff, wherever it is put (Section 2); the loss is a matter of support, not of weights: a weighted objective term discourages such a point, a cutoff removes it. Not everything helps. Where the pump reaches nothing no variant does; a cheap fallback that anchors the cutoff to the rounded point before an incumbent exists is harmful everywhere; and asked for a solution below a stated target, with the FGL pump written from scratch and no objective weighting, the cutoff in the model, with the rounded point checked against that model, is a strong baseline, beaten only at a shallow

target, almost only on mixed problems, and only by the completion (Section 7). Section 3 states the recovery at its two levels and the design principle that makes an invalid constraint in the projection LP legitimate, acceptance being decided by a second LP that carries only the true constraints; Section 4 reads two implementations and lists what we added to one of them; Section 5 describes the factorial study inside SCIP on all MIPLIB 2017 instances with mostly binary variables, on the same instances, seeds and budget, with every solution revalidated on the original model; Section 8 closes with what the setting does not show.

2 The phenomenon, and the room the cutoff leaves

With the cutoff in place the LP minimises a distance to $\hat{x}$ subject to $c^\top x \leq U$. If the projection without the cutoff has $c^\top x > U$ at its optimum, then by convexity every optimum with the cutoff satisfies $c^\top x = U$: the LP iterate sits on the hyperplane. (This fails at the first round, where $c^\top \tilde{x}_0 = z_{\text{LP}} \leq U$ and the constraint is inactive.) Write $\delta = c^\top \hat{x} - c^\top \tilde{x}$ for the amount by which rounding worsens the cost. When $\delta > 0$ the two sequences are separated by the cutoff, $\tilde{x}$ on the hyperplane and $\hat{x}$ strictly above it, and for the rounding to be able to land below the incumbent one needs

$$\delta \leq z_{\text{inc}} - U = (1 - \lambda)(z_{\text{inc}} - z_{\text{low}}) : \quad (3)$$

the rounding gap must not exceed the room the cutoff leaves. This is a heuristic criterion, since lowering U moves $\tilde{x}$ and hence δ , but it separates problem classes before anything is run: on set covering the median of $\delta/(z_{\text{inc}} - z_{\text{LP}})$, with z_{LP} the first-LP value, is +0.62, because rounding up a fractional cover is feasible and always costlier, so (3) fails at $\lambda = 0.5$; on the MIPLIB instances of the same sample the median is close to zero, and the criterion holds for every cutoff short of the incumbent. These and the following measurements come from a from-scratch pump with cutoff, written on Gurobi’s LP solver, on 37 pure-binary instances (16 set covering from OR-Library, 11 from MIPLIB 2003, 10 from MIPLIB 2017), with $z_{\text{low}} = z_{\text{LP}}$ and z_{inc} held fixed at a reference value, so that the census is taken against a cutoff that does not move; they are reported in full in the accompanying material.

We instrumented that pump to test, at every round, whether $\hat{x}$ satisfies the original constraints, cutoff excluded, and if so where it sits relative to U and z_{inc} . Table 1 reports two things. First, the *fraction* of feasible rounded points that lie above the cutoff: median 66% over all instances and the same at every λ tried, carried by the covering instances, where it is two thirds at every level; on the MIPLIB instances it is much more variable and, except in the smallest cell (five instances at $\lambda = 0.25$), lower, because there the rounding gap is small and the rounded point often lands below the cutoff. Second, the *position* in the band $[U, z_{\text{inc}}]$, 0 meaning “on the cutoff” and 1 “on the incumbent”, of the points that are feasible, above the cutoff and better than the incumbent from which the cutoff was built, that is the points whose loss is a real loss: median 0.057 at $\lambda = 0.5$ over the 31 instances that produced such points (5,197 points in all), and on 29 of the 31 the per-instance median lies in the lower half of the band. The same holds at $\lambda = 0.75$ and 0.25. The position depends on how the band is defined, and the aggregation script prints every combination. We report the band $[U, z_{\text{inc}}]$ the cutoff actually carves, with z_{inc} the reference incumbent from which U was built, and the improving points only; pooling all feasible rounded points gives 0.23, and taking as the top of the band the incumbent known at that moment gives 0.57, ten times the reported value, because that band is much narrower.

Over the three levels tested the pooled fraction lost changes little, while the per-family fractions on the MIPLIB instances do move (Table 1). The normalised position moves with the width of the band, as it must, from 0.010 to 0.074, and at every level the lost improving points stay in the lower half of the band on almost every instance: move the cutoff down, the band shrinks, and the improving solutions gather again just above the new level (Figure 1).

Table 1: Feasible rounded points above the cutoff (from-scratch pump). Left: fraction, medians over three seeds within an instance and then over instances. Right: position in the band $[U, z_{\text{inc}}]$ of the points that are feasible, above the cutoff and better than the incumbent (seed 0). In parentheses, the number of instances behind each cell.

class	fraction above U , median			position in $[U, z_{\text{inc}}]$, median		
	$\lambda=0.25$	$\lambda=0.5$	$\lambda=0.75$	$\lambda=0.25$	$\lambda=0.5$	$\lambda=0.75$
set covering	66.6% (16)	66.9% (16)	66.4% (16)	0.007 (16)	0.017 (16)	0.056 (16)
MIPLIB 2003	53.3% (10)	35.7% (11)	27.1% (11)	0.030 (7)	0.107 (10)	0.194 (9)
MIPLIB 2017	71.4% (5)	17.8% (7)	13.6% (9)	0.058 (4)	0.019 (5)	0.039 (6)
all	66.0%	65.9%	64.4%	0.010	0.057	0.074
instances	31	34	36	27	31	31
median < 0.5	–	–	–	26	29	29

The denominators change with λ and with the family, and the per-family medians rest on between four and sixteen instances. Fractions are computed on the instances with at least one feasible rounded point (the excluded ones are always MIPLIB instances: six of 37 at $\lambda = 0.25$, three at 0.5, one at 0.75); positions on the instances that produced at least one lost improving point; the last row counts the instances whose median position lies in the lower half of the band.

Three levels on one seed do not establish scale invariance, and we claim none; what they say is that a policy that only moves the cutoff cannot by itself recover what the cutoff excludes.

Support, not weights. A natural objection is that a cutoff is a constraint, a constraint has a multiplier, and [1] already replaced the cutoff by a weighted objective term

$$\min \frac{1 - \alpha}{\|\Delta\|} \Delta(x, \hat{x}) + \frac{\alpha}{\|c\|} c^\top x, \quad (4)$$

where $\|\cdot\|$ is the Euclidean norm of the coefficient vector of the corresponding linear function, so that everything said about the cutoff should have a Lagrangian counterpart. It does not. The equivalence between a constraint and a penalty holds for the LP; the pump is the composition of an LP with a rounding, and the loss above lives on the rounded points, where no duality operates. Under (4) the polyhedron is unchanged, and a costly rounded point that is feasible for the original constraints remains feasible, merely discouraged. Under (2) it is outside.

Observation 1. Let $\hat{x}$ be feasible for P , and let the round solve the pure projection, $\alpha = 0$ in (4) and no cutoff. Then $\min\{\Delta(x, \hat{x}) : x \in P\}$ has optimal value 0, and every optimal solution coincides with $\hat{x}$ on the variables that enter the distance. On a pure binary problem $\hat{x}$ is the only optimum; on a mixed problem the solver may return another point of the zero-distance face, one that agrees with $\hat{x}$ on B and differs on the continuous variables. Either way the rounded point stays inside the polyhedron the next LP projects onto; among the two objective mechanisms compared here it is the cutoff, and not the weighted term, that puts it outside.

The weighted term does not break this: if $\hat{x} \in P$ and (4) attains a finite optimum over P for every α near zero, then, P being a polyhedron with finitely many faces, there is a threshold $\bar{\alpha} > 0$ below which every optimum still lies on the zero-distance face, and SCIP’s schedule, $\alpha = 1$ multiplied by 0.1 every round, is below it after a few rounds. We do not bound the threshold, and claim only that the weighted term, unlike the cutoff, never removes $\hat{x}$ from the polyhedron.

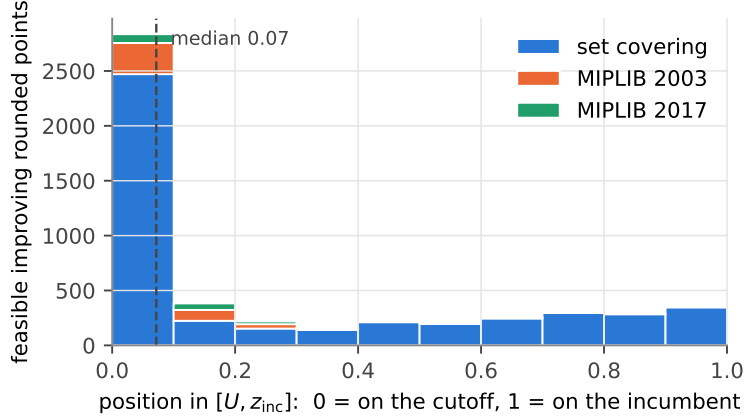


Figure 1: Where the lost improving points sit: position in the band between the cutoff U and the incumbent, $\lambda = 0.5$, all 5,197 points pooled (the dashed line is their median; the median of per-instance medians is 0.057). They pile up against the cutoff.

3 The recovery, and why an invalid constraint is legitimate

All variants below share the same pump and differ in two things: whether the constraint $c^\top x \leq U$ is added to the projection LP, and at which λ ; and what is done with the rounded point once it exists.

Two levels of recovery. The cheap level, which we call the *test*, is the one [13] prescribed for pure-integer problems: at every round, test $\hat{x}$ for feasibility against the original constraints, cutoff excluded, and if feasible offer it to the solver. It is one pass over the constraint matrix, cheap next to the LP re-solve it may save. It is sound but not sharp on a mixed problem, where $\hat{x}$ carries the continuous components of a projection LP that minimised a distance under a constraint that is not part of the problem: as a completion of the rounded integers they are arbitrary, and the test rejects points for which a feasible completion exists. The sharp level, which we call the *completion*, is the postprocessing of [13] as written: fix the integers at $\hat{x}$ and minimise $c^\top x$ over the original constraints. It decides feasibility instead of guessing it and returns the best completion rather than an incidental one; on a two-stage problem, integer first stage and continuous second, it is the whole question. We reimplement exactly that, and what is ours is only the placement: the completion runs on a second LP kept apart from the pump’s own, because the two keep separate bases. Between two rounds only the integer bounds of the second LP move, so it reoptimises with the dual simplex from its own last basis, whereas fixing those bounds inside the pump’s dive would destroy the basis the pump warm-starts from. What this costs in practice is measured, not assumed, in Section 6.

Two decoupled LPs. That the cutoff is excluded from the test is not a detail. The projection LP and the LP that decides feasibility are two different linear programs, and keeping them apart is what makes (2) legitimate at all. The first carries a constraint that is *not valid* for the problem, and nothing in the scheme requires it to be the only one: its role is to be a field of pressure, the means by which the LP trajectory is pushed towards good solutions or away from a point it keeps returning to. The second sees only the true constraints and the true objective; it is the certificate, and correctness rests on it alone. Two properties must be kept apart. *Soundness*: no point that violates the true constraints is ever accepted, whatever the first LP contains. *Effectiveness*: nothing of the sort is guaranteed; a wrong bound or a constraint that excludes part of the feasible region can make the first LP infeasible or stop it

from producing candidates. What the decoupling buys is the freedom to try such constraints without risking a wrong answer, not a promise that they help. (Constraints of the second kind, $\Delta(x, \hat{x}_i) \geq \Delta(\tilde{x}_i, \hat{x}_i)$ added permanently, are used in the variable neighbourhood pump of [19, 20]; local branching [14] is the inequality reversed.) In this paper the cutoff is the only invalid constraint we use.

Cutoff level, and restart. U is (2) with λ chosen once, z_{low} read from the solver’s dual bound when the heuristic is entered, and U recomputed at every round so that it follows the incumbent down; when the pump finds a solution it does not stop but updates z_{inc} and continues, without which the cutoff would act at most once. Which λ is not obvious: at $\lambda = 1$ nothing improving is excluded, though the constraint still constrains the projection; at small λ the pressure is strong and by (3) most rounded points are lost. We treat λ as the knob it is and measure a range of levels rather than defend one. A band of width $w(z_{\text{inc}} - z_{\text{low}})$ below the incumbent is sometimes a more natural way to say the same thing, but it is the same thing, (2) at $\lambda = 1 - w$, and we do not present it as a separate policy. In our harness $z_{\text{low}} = z_{\text{LP}}$ (Section 5), so $U \geq z_{\text{LP}}$, the first LP optimum satisfies the constraint, and the projection LP is never infeasible; the fallback of the next paragraph is the one setting in which it can be.

Before the first incumbent. Equation (2) needs an incumbent, and before the first one exists the constraint is free, precisely on the instances where the pump stalls. The cheap way to have a cutoff from the second round on is to fall back on the cost $c^\top \hat{x}$ of the current rounded point as the upper reference until an incumbent exists. It is tempting, and the objection to it is substantive: the rounded point is not a solution, its cost bounds nothing, and since $\hat{x}$ need not lie in P the constraint $c^\top x \leq c^\top \hat{x}$ can make the projection LP infeasible. The design of Section 5 treats this fallback as a factor of its own, never bundled with anything else, so that its effect can be read alone.

4 What implementations do, and what we added

We read the two open-source implementations rather than the papers. SCIP’s pump is the file `heur_feaspump.c`; at the development snapshot we pin (commit `dba4b2a`, version string 11.0.0, with SoPlex at commit `f0dbc81`) it declares itself “objective feasibility pump 2.0”: it is the objective-weighted pump of [1], with the round-and-propagate stage of [16] available as an option that is off by default. It contains exactly one call that offers a solution to the solver, made on the LP iterate in the branch taken when it has no fractional variable. The rounded point is built every round, used for distances and cycle detection, and never tested for feasibility, neither directly nor by the postprocessing LP. SCIP imposes no cutoff on the diving LP either: the objective enters only through α in (4). Of the two coupled parts of the original design this implementation keeps neither, and a user who adds the cutoff to the model gets the black-box use of Section 1. CBC’s `CbcHeuristicFPump.cpp` [17] does the opposite: its rounding routine returns success when the largest constraint infeasibility is within tolerance and the caller keeps the point, and when the point is integral it fixes the integers and re-solves the LP on the continuous variables. “The pump does not look at the solutions it produces” is thus a fact about one implementation, not about the method.

Our experiments add to SCIP’s pump the cutoff (parameter `cutlam`) and its fallback (`cutfallback`), the feasibility test of the rounded point (`tryrounded`) and the postprocessing on a clone LP (`lpfix`), the restart (`restartonsol`), a stop after the call (`stopafter`), a treatment-blind delivery of integral iterates, and the completion counters of Section 6. All default to off, so that every variant is the same binary, differing only in parameter values.

5 Experimental setup

The pump as a standalone heuristic. Inside a full branch-and-cut the pump is called once at the root (Section 8). To measure the pump rather than the node we run SCIP at the snapshot above with `limits/nodes = 1`, all other primal heuristics off, the pump set to run at every opportunity, and `stopafter`, so that the clock and the solution belong to the heuristic alone. Presolving is left at its defaults, and every solution is re-validated on the original, unpresolved model (below). In this harness no cutting plane is ever separated: the pump is reached after the first LP and the solve ends when it returns, and across the campaign logs the separators record zero calls and zero applied cuts. The polyhedron the pump projects onto is therefore the plain LP relaxation of [13], and the anchor z_{low} of (2) is the first-LP value z_{LP} , fixed for the whole run.

Why the relaxation is left unseparated. This is not a neutral detail. A cut is valid for the integer hull of the model it was separated from, and once a solver holds a finite primal bound that model is the problem restricted to improving solutions: SCIP’s aggregation separator tries the objective constraint with the cutoff bound as right-hand side, and root reduced-cost fixing tightens global bounds with the incumbent. A feasible but non-improving point may then be excluded by a cut or a box. For a paper about which feasible rounded points a cutoff makes invisible that would be a confound in the instrument, and the unseparated relaxation removes it. Anyone embedding these variants in a full solve inherits it: soundness is not at risk, since acceptance is decided by the constraint handlers, but a failed completion then means “no completion in the restricted polyhedron”. Presolve is a milder instance of the same point: its dual reductions preserve an optimum, not the feasible set, so the model the pump works on can lack feasible points of the original; this is why every solution is re-checked on the original model.

Time limit and reference. Each instance gets a time limit of $20\times$ the time of its own first LP, clamped to $[20, 300]$ seconds, so that instances of very different size get comparable numbers of pump rounds rather than comparable seconds. Of the 424 instances analysed, 231 sit at the floor of 20s, 122 get the budget the rule dictates, and 71 sit at the cap. The first-LP value z_{LP} is also the reference of the primal integral [4]: the same for every variant of an instance, and requiring no optimum. The campaign ran on two sets of four-core blades with hyper-threading disabled, Xeon E5-2623 v3 at 3.00 GHz and Xeon E3-1220 v2 at 3.10 GHz. An instance never straddles: every variant and every seed of it runs on one blade, and its time limit is derived from a first LP timed on that blade, so the budget normalises itself. On ten first LPs run on both, with identical iteration counts, the older blades are slower by 1 to 12%, 6% in the median. The 61 runs of an instance execute one after another on a blade allocated exclusively to them, the variants in a random order within each seed. All comparisons reported here are paired within an instance.

Instances. All 463 instances of MIPLIB 2017 [18] with at least 80% binary variables are run, each through every variant and every seed; nothing is selected on outcome. The remaining variables are continuous or general integer, and the pump handles the latter as SCIP does. The value and time of the first LP are read once per instance by a probe with separation off and a limit of 600s. Thirty-seven instances do not solve their first LP within it and are excluded before the experiment begins; two more (`supportcase1`, `neos-3135526-osun`) lose part of their runs to an LP numerical failure inside SCIP and are excluded because a paired design with empty cells is no longer paired. On `supportcase1` the failure is not independent of the treatment: every run that carries the cutoff constraint fails, in all nine such variants and on all five seeds, and none of the runs without it does. Excluding the instance therefore removes one

instance on which the cutoff, as implemented, fails outright, and we count it as such. All 39 are listed in the supplementary material. The testbed is therefore 424 instances, 12 variants and 5 seeds, a complete grid of 25,440 runs. Every solution a run reports is written to a file and re-checked by a separate SCIP process on the original model: of the 10,455 solutions reported none is infeasible, and the recomputed objective agrees with the reported one on all but six runs of one variant on two instances, where the recomputed value is *better*, the rounded point having carried integer variables at $1 - 10^{-15}$ whose image in the original space is worth slightly less. No reported value is better than what its file contains.

Two strata, not a filter. The plain pump, no cutoff and no recovery, is run first on every instance at half the budget the variants get (floored at ten seconds) and on one seed, as a *pilot*. Its outcome stratifies the testbed: on the 176 instances where the pilot finds a solution (stratum E1) the question is how much better each variant does; on the 248 where it finds nothing (stratum E2) the question is whether anything is found at all. The two strata are thus defined a priori, by that single run of the pilot, and the whole design, factors, levels, endpoints and the twelve comparisons of Table 2, was fixed before any result of the variants was looked at (two further comparisons were added later and are reported as such). The pilot gates nothing: every variant runs on every instance, and the strata only decide which question a result answers. The pilot is the method being challenged, so E1 is exactly the ground where that method already works; half the budget is enough for the plain pump on those instances, so the stratum is not made small by a stingy pilot. The pilot is a single run at half budget while every variant is judged on the median of five seeds at full budget, so the strata and the endpoint live at different noise levels, and the two can disagree in either direction on a few instances (Section 6).

Variants. The design has two factors: what is done with the rounded point, nothing, the test, or the completion; and where the cutoff sits, absent or (2) at $\lambda \in \{0.5, 0.75, 0.9, 0.95\}$. Ten of the fifteen cells of the grid are run: no recovery at $-$ and 0.5 (*none₋*, *none_{0.5}*), where *none₋* is the objective-weighted pump of [1] exactly as SCIP implements it, run unchanged, and is the literature baseline of this study; the test at all five levels (*check₋* to *check_{0.95}*); the postprocessing at $-$, 0.5 and 0.95 (*FGL₋*, *FGL_{0.5}*, *FGL_{0.95}*). The subscript is λ and $-$ means no cutoff constraint at all. *none_{0.5}* is the cell closest to the black-box use of the cutoff, a user of the stock implementation adding the constraint and nothing else, with two differences that should be kept in mind. The practitioner has an incumbent from outside and the constraint is active from the first round, whereas here no incumbent exists at the start and the constraint becomes active only once the pump has found a solution of its own; and the practitioner's U is fixed and the pump stops at the first solution of the restricted model, whereas here U follows the pump's own incumbents and the pump goes on. The truly one-shot scheme, external U held fixed, is the one the from-scratch census of Section 2 runs. *FGL_{0.5}* is the scheme of [13] as written. Two further variants add the fallback of Section 3 at $\lambda = 0.5$ (*none_{0.5}^{fb}*, *check_{0.5}^{fb}*). All twelve are the same binary and keep SCIP's default weighting (4), which is the same in every cell and is not a treatment: every variant, *FGL_{0.5}* included, is the pump of [1] with the parts of [13] added to it, the two designs being additive rather than alternatives. Since α starts at 1 and is divided by ten every round, after the first few rounds every variant is, in effect, a pure-distance pump.

Solution delivery is treatment-blind. The stock pump hands its solution to the solver only after the loop, on the integral iterate that stopped it. With the restart the loop does not stop, so a variant with no recovery would report a solution only when the clock ran out while a variant with the test would report every integral iterate as it passed, confounding the

treatment with the bookkeeping. We therefore hand in every integral iterate the moment it appears, at the top of the loop and before any variant-specific code runs, in every cell alike; the median time to the first solution is between 4.0 and 4.6 seconds for all ten variants of the grid in E1.

Seeds, aggregation, metrics. Five seeds per (instance, variant), the same five values for every variant, so that runs are paired by (instance, seed). The unit of inference is the instance: for each variant the median over its five seeds is taken first, comparisons are paired per instance on that median, a tie is a difference below 10^{-6} , and the test is a sign test on the non-tied instances. Note that the median over seeds can be 100 on an instance where some seeds of a variant do find a solution: that is the aggregation, not the endpoint. Twelve comparisons were declared before the data were looked at (the rows of Table 2) and are repeated in the two strata; Holm’s correction [15] is applied within each stratum, two families of twelve.¹ Under the more conservative reading of a single family of twenty-four, the corrected p of the seventh row of Table 2 becomes 0.013 and that of the sixth 0.26, and nothing below changes sign. Since the strata are defined by a run of one of the variants, we also ran the twelve comparisons unstratified on all 424 instances: every sign is the same, with E2’s ties added, the completion against the test at mid-gap is 28–387–9 (Holm $p = 0.010$), loose against mid-gap 67–312–45 ($p = 0.14$), and the two fallback comparisons 21–308–95 and 22–302–100 ($p < 10^{-4}$). Two further comparisons, $\text{FGL}_{0.5}$ against none_- and against $\text{none}_{0.5}$, were added after the design as the paper’s headline (Section 6); they are reported without correction, which at their p -values changes nothing. Counts per run are reported where they add information, but only descriptively, since runs on the same instance are not independent. The primary endpoint in E1 is the final gap to the LP bound, $100|z - z_{\text{LP}}|/\max\{|z|, |z_{\text{LP}}|\}$, charged at 100 to a run that finds no solution, at 100 when z and z_{LP} have opposite signs, and at 0 when both vanish, so that it is bounded by 100 and no solution scores worse than none; secondary are the primal integral divided by the horizon, the time to the first solution, rounds, LP iterations, and for the completion variants the number of completion LPs solved and how many produced an accepted solution. In E2 the endpoint is found or not found, reported for each pair of variants as a 2×2 table, because the “neither” cell is where most of the mass is.

6 The result: FGL’s scheme restored inside SCIP

Table 2 reports, on the 176 of the 424 instances that form stratum E1, the twelve comparisons declared in advance, and Table 4, in Appendix A, the twelve variants one by one. The headline comes first; the rest of the section takes it apart into the factors of the design, and the last paragraph prices them.

The headline. The objective-weighted pump of [1] as SCIP runs it, with the cutoff at mid-gap and the completion of [13] added to it, against that pump alone: $\text{FGL}_{0.5}$ beats none_- on 116 instances, ties on 51 and loses on 9, lowering the median final gap from 59% to 30%, with a median paired difference of 13 points where the two differ. Against the black-box cutoff, $\text{none}_{0.5}$, the same scheme wins 87–75–14 with a median difference of 3.3 points, and by at least ten points on 34 instances. Against the test without a cutoff, the cheapest sound thing one can do, the scheme of [13] as written, $\text{FGL}_{0.5}$, wins 114–10. The first two comparisons are not among the twelve declared in advance, and we report them without correction: at $p < 10^{-4}$

¹What the corrected p means for the reader: below 0.05, the chance that even one of the twelve claims of the family is false is under one in twenty, and the comparison can be taken as established; between 0.05 and 0.10 it is a trend, and is called so; above that it is a null.

comparison (A vs. B)	instances			p -value		median diff. $B - A$, non-tied
	A better	tie	A worse	sign test	Holm	
check ₋ vs. none ₋	22	154	0	$< 10^{-4}$	$< 10^{-4}$	+17.1
check _{0.5} vs. none _{0.5}	67	93	16	$< 10^{-4}$	$< 10^{-4}$	+2.8
none _{0.5} vs. none ₋	90	72	14	$< 10^{-4}$	$< 10^{-4}$	+11.0
check _{0.5} vs. check ₋	105	59	12	$< 10^{-4}$	$< 10^{-4}$	+12.7
check _{0.95} vs. check ₋	119	50	7	$< 10^{-4}$	$< 10^{-4}$	+9.8
check _{0.95} vs. check _{0.5}	64	73	39	0.0176	0.0528	+0.7
FGL _{0.5} vs. check _{0.5}	27	142	7	0.0008	0.0033	+1.7
FGL _{0.95} vs. check _{0.95}	18	147	11	0.2649	0.2649	+0.1
FGL ₋ vs. check ₋	12	160	4	0.0768	0.1536	+1.4
FGL _{0.5} vs. check ₋	114	52	10	$< 10^{-4}$	$< 10^{-4}$	+13.3
check _{0.5} ^{fb} vs. check _{0.5}	18	73	85	$< 10^{-4}$	$< 10^{-4}$	-7.6
none _{0.5} ^{fb} vs. none _{0.5}	19	67	90	$< 10^{-4}$	$< 10^{-4}$	-17.7

Table 2: E1, the twelve comparisons declared before the data were looked at, paired per instance on the final gap: instances on which the first variant is better, tied, worse; sign test on the non-tied; Holm correction over the twelve; and the median of the paired difference in gap points over the non-tied instances (positive means the first variant is better).

no correction matters. What the rest of this section does is take that headline apart into its factors.

The black-box cutoff already helps, and the recovery makes it help more. Adding the cutoff at mid-gap to a pump that ignores its rounded points, none_{0.5} against none₋, improves the final gap on 90 instances and worsens it on 14, and lowers the median gap from 59% to 40%. So the black-box use is not useless; it is wasteful. Testing the rounded point never hurts without a cutoff, check₋ against none₋ 22–0; with the cutoff at mid-gap the same test is worth much more, 67–16 for check_{0.5} against none_{0.5}, and the median gap drops from 40% to 31%. This is the interaction Section 2 predicts: without a cutoff a feasible rounded point is recovered by the projection anyway (Observation 1) and testing it only saves time; with a cutoff it is outside the polyhedron, and looking at it explicitly, by the test or by the completion, is the only way to keep it. The test can also cost something, on 16 instances here, because an accepted solution lowers U at once and moves the trajectory. At equal recovery the cutoff is a lever, check_{0.5} against check₋ 105–12 and check_{0.95} against check₋ 119–7, and the diagnostics show why: without a cutoff the pump performs eight thousand rounds for two thousand LP iterations in all, barely a re-solve per round, wandering; with one, eight hundred rounds for a hundred thousand iterations, each projection a real re-optimisation under the pressure of the constraint.

The LP recovery improves on the test where the cutoff is tight, and only there. At $\lambda = 0.5$, re-solving the continuous variables on the original constraints instead of accepting the rounded point as it stands, FGL_{0.5} against check_{0.5}, is better on 27 instances, tied on 142 and worse on 7, Holm-corrected $p = 0.003$. At the loose level the postprocessing adds nothing measurable (FGL_{0.95} against check_{0.95}, 18–11, $p = 0.26$) and without a cutoff it adds little (FGL₋ against check₋, 12–4, $p = 0.15$). The counters of Table 4 say why. Without a cutoff almost every completion LP is feasible (99.4% of 8.4 million) and almost none is accepted (242), because the projection recovers the point anyway (Observation 1) and the completion has nothing to add. With the cutoff at mid-gap 58% of the completion LPs are infeasible, the fixed integers being off the polyhedron altogether, but the ones that are feasible are accepted eighty times more often (19,865), because they are points the projection would have discarded. The effect also sits exactly where it should. Of the 176 instances, 77 have continuous variables

after presolve and 99 have none; on the 99 pure binary instances no completion LP is ever built and the two variants are the same code path, so they return the same final gap on 98 (on the remaining one the clock stopped two identical runs at different rounds), and on the 77 mixed instances the comparison is 27–44–6. The gains have a median of 3.3 points of gap and reach 17; the seven losses, six of them on mixed instances, have a median of 0.6 and none exceeds 4. Splitting E1 at its median nonzero count, the comparison is 12–3 on the small half and 15–4 on the large one. Neither split is in the declared family: the mixed/pure split is the prediction of the mechanism, the size split is exploratory, and both are reported with raw sign-test p (0.0003 on the mixed instances, 0.035 and 0.019 on the two halves) and no correction.

The level. A single comparison between levels was declared, the loose cutoff against mid-gap: $\text{check}_{0.95}$ against $\text{check}_{0.5}$ is 64–73–39 with Holm $p = 0.053$, a trend and not a result. The four levels with the test have median gaps of 31%, 30%, 28% and 29% at $\lambda = 0.5, 0.75, 0.9, 0.95$, but that ordering is one of marginal medians, not a paired comparison, and no test supports it. We read the levels as interchangeable within the resolution of this design and recommend none.

What the completion costs. The completion is not free and Table 4 measures it: the three completion variants solve between five and eight and a half million completion LPs each over their 880 runs, six to ten thousand per run. Without a cutoff almost none is useful; with the cutoff at mid-gap 3.1 million of the 5.3 million solved at mid-gap are infeasible outright, because fixing the integers of a point the cutoff has pushed off the polyhedron often leaves no continuous completion at all. Each call is a dual re-solve on a fixed face, and it costs throughput rather than latency: the median time to the first solution moves by less than 0.1 s, while the median number of rounds falls by 6 to 13% in all three pairs of Table 4 (7,695 $\rightarrow$ 6,720, 834 $\rightarrow$ 780, 944 $\rightarrow$ 820). It is called every round, and its yield is a function of the cutoff. The counters count every call, including the many on rounded points already seen, so their ratios are meaningful and their absolute values are not. Finally, on 23 of the 176 instances all twelve variants return the same final gap: five solved to the LP bound by everyone, eight on which the median seed of every variant is left at gap 100 although the pilot’s single run, and some seeds of several variants, do find a solution, and ten agreeing on an intermediate value. The policies do nothing on one instance in eight.

7 Where it does not help

E1 and E2 are two strata of the same pre-declared design, and the twelve comparisons are repeated in both; this section reports where that design pays nothing, and then a separate experiment that changes the pump and the endpoint, including one endpoint on which the completion does win, and one on which the plain cutoff is hard to beat.

Where the pump finds nothing (stratum E2). Stratum E2 is the 248 instances on which the plain pump at half budget finds nothing, and on it the design has no effect we can detect. The ten variants of the grid find a solution on between 22 and 27 instances on at least one seed, the two fallback variants on 11 and 9 (Table 5, in the appendix). Paired per instance, every one of the twelve comparisons is null: the largest count of decisive instances in either direction is ten, and after Holm every p -value is 1 (the per-instance counts are not tabulated). On 230 of the 248 instances all twelve variants agree, almost always on finding nothing. Per run (Table 6) the four cells read, for the ten variants of the grid: only one of the two finds a solution, from none to a couple of dozen runs; both, between fifty and eighty;

neither, always above eleven hundred of 1,240. The cutoff moves a few runs in each direction with no consistent sign, and the two recoveries move almost none. A null on this stratum is a statement of precision, not of absence: with about fifteen discordant instances per comparison the design can only exclude large effects. For the cutoff without recovery the discordant instances are 9 against 6, whose exact 95% interval on the proportion is $[0.32, 0.84]$, so odds ratios between 0.5 and 5 remain compatible with the data. This is where a first solution is hard, and neither recovery, both of which act on what happens *after* a rounded point turns out feasible, has anything to act on: the pump does not produce feasible rounded points, so there is nothing to keep or complete; the completion variants solve 3.2 million completion LPs each in E2 and between 28 and 57 yield an accepted solution, one in sixty to a hundred thousand. Whether a cutoff can be made to help *find* a solution rather than improve one is a question this design does not answer, and the honest reading of Tables 5 and 6 is that on the hard half of the testbed it does not.

The fallback. Anchoring the cutoff to $c^\top \hat{x}$ before an incumbent exists (Section 3) is the one intervention that hurts, with and without the test. In E1, $\text{check}_{0.5}^{\text{fb}}$ against $\text{check}_{0.5}$ is 18–73–85 and $\text{none}_{0.5}^{\text{fb}}$ against $\text{none}_{0.5}$ 19–67–90, both $p < 10^{-4}$. The two fallback variants also report no solution on 58 and 71 of the 176 instances, which no other variant does on more than three (Table 4): per instance, the fallback loses a solution on 55 and 68 instances respectively and gains one on none. In E2 it is the only factor that moves the found-or-not counts, losing a solution on 16 instances and gaining one on 2 to 4 (Table 6). A cutoff needs a bound it can trust, and the cost of a rounded point is not one.

A clean pump on Gurobi, and a stated target. Two questions remain open in Section 6: whether the findings depend on SCIP’s pump, which carries the objective weighting of [1] and everything else a solver adds, and what happens in the situation the cutoff was designed for, a user who holds a solution and wants a better one *by a stated amount*. A separate experiment, E3, answers both with the FGL pump written from scratch on Gurobi, pure distance and no objective term, rounding to the nearest integer, with the perturbations and restart rule of [13] as published ($T = 20$, the number of components flipped when the rounding repeats being uniform in $[T/2, 3T/2]$; a cycle window of three roundings; a restart forced every 100 rounds, flipping every binary j with probability $0.2 + |\tilde{x}_j - \hat{x}_j|$), one thread, dual simplex, and the plain LP relaxation with no cutting plane (code, lists and raw results in the supplementary material, `fp_gurobi/`). A *pilot* run of the same pump, with the completion and its own seed, provides the starting solution z_{inc} , z_{best} is the best value known to MIPLIB 2017, and the target is

$$U = z_{\text{best}} + a(z_{\text{inc}} - z_{\text{best}}), \quad a \in \{0.5, 0.9\}, \quad (5)$$

closing half of the gap to the best known value, or one tenth of it. U is fixed for the whole run, no policy knows z_{best} , and every policy stops at the first point that is feasible for the original constraints with $c^\top x \leq U$; the endpoint is whether it gets there within the budget, and when. Three policies use the same pump. NAIVE adds the constraint $c^\top x \leq U$ to the model before the pump and checks every rounded point against that model, constraint included: it is the cutoff as a user imposes it, with the strongest check a pump can make without looking at the original constraints on their own (the black box of Section 4 does not even test the rounded point). TEST and COMPLETION keep the model intact and put an *inner* constraint $c^\top x \leq U'$, $U' = U - w(U - z_{\text{LP}})$, in the projection LP only, recovering every rounded point on the original constraints, by the feasibility test and by the LP with the integers fixed respectively; the margin w is meant to catch, below U , the feasible roundings that Section 2 finds just above the constraint, it is the only tuning knob, and NAIVE is its $w = 0$ point. Every accepted point is re-checked on the original model with all variables fixed to it, at Gurobi’s absolute

Table 3: Outcome of the target experiment on the evaluation set. Every instance is run with five random seeds under each policy. An instance counts as *solved* when at least three of its five seeds reach a feasible point with $c^\top x \leq U$ within the time limit (the outcome is the median over seeds of the success indicator); *pairs* is the number of (instance, seed) pairs that reach such a point, out of the $5N$ available. The target is $U = z_{\text{best}} + a(z_{\text{inc}} - z_{\text{best}})$ and w is the width of the inner margin as a fraction of $U - z_{\text{LP}}$: policy NAIVE imposes a static cutoff constraint in the model, while TEST and COMPLETION impose an inner constraint at $U' = U - w(U - z_{\text{LP}})$ and recover a point for the true target by a direct feasibility test and by FGL’s LP completion, respectively. Subsets: benchmark instances (the primary test set) and non-benchmark ones; pure 0–1 instances ($n_{\text{cont}} = 0$) and mixed ones ($n_{\text{cont}} > 0$).

Subset	N	NAIVE		TEST		COMPLETION	
		solved	pairs	solved	pairs	solved	pairs
$a = 0.5$ ($w = 0.02$ for TEST, $w = 0.15$ for COMPLETION)							
all	134	87	438/670	84	424/670	85	430/670
benchmark	43	28	143/215	26	130/215	28	142/215
non-benchmark	91	59	295/455	58	294/455	57	288/455
pure 0–1	70	56	278/350	52	268/350	50	254/350
mixed	64	31	160/320	32	156/320	35	176/320
$a = 0.9$ ($w = 0.05$ for TEST, $w = 0.05$ for COMPLETION)							
all	134	104	519/670	103	518/670	112	561/670
benchmark	43	32	162/215	32	163/215	35	179/215
non-benchmark	91	72	357/455	71	355/455	77	382/455
pure 0–1	70	62	313/350	63	315/350	63	314/350
mixed	64	42	206/320	40	203/320	49	247/320

tolerances; one success in 2,891 failed that check and is counted as a failure. Of the 463 instances of Section 5, 430 have a best known value, and a reconnaissance run of the pilot leaves 154: excluded are the instances with general integer variables or non-linear constraints, which the 0–1 pump does not handle, those on which the pilot finds nothing in 150s or is already at the best known value, and those whose first LP exceeds 5s. The split is exogenous: the 43 instances of the MIPLIB 2017 Benchmark Set are the primary test set, 20 instances drawn at random outside it tune w , and the remaining 91 are the secondary test set; 70 of the 134 are pure 0–1 and 64 have continuous variables. Time limit as in Section 5, here between 20 and 100s, five seeds (0–4; 10–14 for the tuning), the three policies of a seed running simultaneously, one thread each, on a blade allocated to them alone: 4,020 evaluation runs. On the tuning instances a rule declared beforehand picks, among seven widths from 0.02 to 0.50 and the $w = 0$ point that NAIVE provides, the w with most successful (instance, seed) pairs, ties broken by the median censored time: at $a = 0.5$ it picks $w = 0$ for TEST, every positive width losing pairs to NAIVE, and $w = 0.15$ for COMPLETION (69 of 100 pairs against 67, within the noise of 20 instances), and TEST is evaluated at $w = 0.02$, the best positive width, as an exploratory arm; at $a = 0.9$ it picks $w = 0.05$ for both (79 and 86 pairs against 76).

Table 3 gives the outcome, and Table 7, in the appendix, the paired comparisons on the outcome alone, with the time kept apart. At the deep target, $a = 0.5$, the band buys nothing: the three policies solve 87, 84 and 85 of the 134 instances, every unstratified comparison on the outcome is a tie on more than 115 instances (9–7–118 for NAIVE against COMPLETION), and where two policies both succeed the one without the extra LP is the faster. At the shallow target, $a = 0.9$, the completion behind a narrow margin never loses to NAIVE on the outcome and wins on 8 instances of 134, seven of them mixed (0–8–126, exact $p = 0.008$ on the

unstratified set; 0–7–57 on the 64 mixed instances, 0–1–69 on the 70 pure ones), paying its extra LP per round in time on the pure instances (38–15 for NAIVE on the instances both solve). The declared family of twelve tests, two values of a by two test sets by three comparisons, contains nothing that survives Holm: on the primary set the completion’s advantage is 0–3–40, $p = 0.25$, and the unstratified count is reported for information. The TEST policy with a margin is never distinguishable from NAIVE, at either target, and at no width tried on the tuning set. With the objective weighting removed and the endpoint changed from “a better solution” to “a solution below U ”, the cutoff in the model is thus a strong baseline: it does what the recovering variants do at the deep target, and it is beaten only at the shallow one, almost only on mixed problems, and only by the completion, which recovers points the feasibility test cannot see, exactly where Section 6 found it useful. The mechanism of Section 2 is visible in the failures: when a recovering policy fails, in 59 to 93% of the runs, depending on policy and target, the pump has produced no feasible rounded point at all, above or below U ; when it has, the best of them sits above the target by a median 20 to 35% of the gap $U - z_{\text{LP}}$ at the deep target, beyond the width the rule selected, and by 5 to 9% at the shallow one, about the width selected; widening the margin moves U' towards z_{LP} , where the pump stops producing feasible roundings at all, and since the relative gap $(U - z_{\text{LP}})/|U|$ spans four orders of magnitude across the 134 instances, no single w fits. What E3 does not change is the recommendation: at the shallow target the recovery never loses on the outcome, at the deep one it is within noise of the baseline, and it is what makes an inner constraint legitimate.

Pressure of another kind. Since an inner constraint need not be valid, we also tried constraints that press towards feasibility rather than cost, on the same 134 instances, five seeds, four policies per seed, with a success rule declared before the runs: a cardinality bound, a sign-only cutoff, no-good constraints on failed roundings, an adaptive U' , and a local-branching ball [14] of radius one tenth of the binaries around the pilot solution, doubled when the pump stalls. Alone, the feasibility constraints are harmful: the objective constraint is what the pump needs. The ball on top of the objective constraint is the one addition that lost to the completion behind a fixed band on at most four instances and gained on up to eleven, 110 instances solved against 107 at $a = 0.9$ and 92–93 against 85 at $a = 0.5$, at the price of more rounds where both succeed; no comparison met the declared rule after Holm’s correction, and we report it as a lead, not a result.

8 Conclusions

Adding an objective cutoff to the model and running the Feasibility Pump on it as a black box is a common practice, and it is not useless: on the stratum of MIPLIB 2017 where the pump already reaches a solution it improves the final gap on 90 instances of 176 and worsens it on 14. But it is wasteful in a specific way. In the from-scratch sample of Section 2, on covering problems two thirds of the feasible rounded points fall above the cutoff, and a smaller share elsewhere; the lost improving points tend to sit just above the cutoff, wherever it is placed, and a pump that does not look at them never sees them again, while a pump that fails to reach a target finds nothing near it (Section 7). [13] said so and paired the cutoff with a recovery; many implementations dropped both. Restoring the test of the rounded point never hurts without a cutoff (22–0) and wins 67–16 over the black box at mid-gap. Restoring the completion, the LP with the integers fixed and the cutoff removed, improves on the test on 27 instances against 7, out of the 77 mixed instances on which it can act at all, with a corrected p of 0.003; at a loose cutoff or without one it adds nothing, which is what the geometry predicts. Added to the objective-weighted pump of [1] as SCIP runs it, the two parts of the FGL scheme beat that pump alone on 116 instances of 176 and lose on 9, and lower the median final gap

from 59% to 30%; against the cheapest sound alternative, the test without a cutoff, the scheme wins 114–10. Where the pump reaches nothing no variant helps, and the cheap fallback that anchors the cutoff to the rounded point before an incumbent exists is harmful everywhere. Asked for a solution below a stated target, with the FGL pump written from scratch and no objective weighting, the cutoff placed in the model, with the rounded point checked against that model, turns out to be a strong baseline: the recovering variants match it at a deep target, and at a shallow one the completion beats it on eight instances of 134, seven of them mixed, 0–8–126 on the outcome, without losing one. The recommendation is not a new heuristic but an old one, restored: keep the cutoff, and keep the recovery that goes with it.

What this does not show. Everything above is measured with the pump as a standalone heuristic. Inside a full branch-and-cut the pump is called once at the root, against a battery of cheaper heuristics and a bound that node processing improves anyway; the ceiling on what a modification of the pump can contribute there is set by how often it runs, not by how well. Our results say what the cutoff and the recovery do to the pump; they do not say that a solver will run faster, and we have not run the embedded experiment. E1 is the stratum the plain pump can already handle, hence the ground most favourable to it, and its instances are smaller than the collection they come from, a median of about 72,000 nonzeros against 110,000 over all 463; every claim of Section 6 is a claim about the stratum, and the five seeds enter the analysis only through the median, whose spread within an instance is large (Table 4: `none` finds a solution on every instance but on 856 of 880 runs). The design compares recoveries at fixed cutoff levels and does not search for a good level; the completion is measured as a count of LPs and by what it does to the time to the first solution and the number of rounds, not by wall-clock per call; the objective weighting inside the pump is at one setting, SCIP’s default, which Section 2 argues should not matter but which was not varied; and MIPLIB 2017 with mostly binary variables is one collection, on which the mechanism is strongest on covering-like structure, where the right reference is not the pump but the Lagrangian heuristic of [11], against which nothing here is measured. E3 tunes one width per policy and per target on 20 instances, where a difference of two or three pairs in a hundred is noise; it reports TEST at $a = 0.5$ at a width the rule did not pick; its sample is the third of MIPLIB 2017 on which a pure 0–1 pump with no help finds a first solution within 150s, so its instances are the easier ones, with a median of 5,800 binaries against 1,700 and 15,000 at the quartiles; the target of (5) is defined from the best value known to the library, which the policies never see but which fixes how deep the target is, and the two values of a bracket what a user’s own choice of U can look like; and its times are relative costs under a common load, compared only between policies that both succeed on an instance, not wall-clocks per call.

Reproducibility. Each table carries in the source the exact command that regenerates it from the raw result files; the aggregators use only the Python standard library and take the result file as an argument. What accompanies the paper is listed in the declarations below.

Statements and Declarations

Funding. No specific funding was received for this work.

Competing interests. The author declares no competing interests.

Data and code availability. The SCIP patches, job scripts, instance lists, raw SCIP logs, the solution files written by every run together with their independent validation on the original models, the from-scratch pump of Section 7 with its job scripts, instance lists and raw result

files, and the scripts that regenerate every table and figure are provided as supplementary material and will be archived with a persistent identifier upon acceptance.

Use of AI tools. An AI assistant (Claude, Anthropic) was used, under the author’s direction and review, to draft and edit text, to write and debug the experimental scripts, and to run an internal adversarial review of the manuscript; every claim, number and line of code was checked by the author, who takes full responsibility for the content.

A Tables by variant

The four tables of this appendix report the variants of Sections 6 and 7 one by one, the per-run counts behind their paired comparisons, and, for the target experiment, the paired comparisons behind Section 7; the text refers to them where their columns are read.

	with a solution		medians					completion LPs		
variant	inst.	runs	gap (%)	PI/TL (%)	t_{first} (s)	rounds	LP iter.	calls	feas.	infeas.
none _—	176/176	856/880	59.12	71.0	4.07	8 032	2 030	—	—	—
none _{0.5}	173/176	840/880	39.97	50.2	4.61	977	104 926	—	—	—
check _—	176/176	856/880	58.87	66.7	3.99	7 695	2 124	—	—	—
check _{0.5}	173/176	840/880	30.82	46.4	4.12	834	105 064	—	—	—
check _{0.75}	173/176	838/880	29.69	44.0	4.04	836	96 620	—	—	—
check _{0.9}	173/176	840/880	27.96	43.9	4.11	873	86 760	—	—	—
check _{0.95}	173/176	840/880	29.39	44.8	4.09	944	80 909	—	—	—
FGL _—	176/176	856/880	55.81	65.3	4.03	6 720	2 124	8 413 518	242	50 873
FGL _{0.5}	173/176	840/880	30.41	44.0	4.15	780	107 402	5 316 978	19 865	3 072 971
FGL _{0.95}	173/176	840/880	29.39	44.2	4.17	820	80 352	5 559 625	6 599	432 887
check _{0.5} ^{fb}	118/176	533/880	51.48	70.7	4.81	248	107 649	—	—	—
none _{0.5} ^{fb}	105/176	466/880	95.84	98.5	6.22	232	108 320	—	—	—

Table 4: E1, the 176 instances of 424 on which the pilot finds a solution, 5 seeds each. Median gap and PI/TL are medians over instances of the per-instance median over seeds; t_{first} , rounds and LP iterations are medians over runs. *inst.* counts the instances on which at least one of the five seeds finds a solution, *runs* the runs that do. The last three columns count, for the completion variants, the completion LPs solved (*calls*), those whose solution was accepted, and those found infeasible.

References

- [1] T. Achterberg, T. Berthold. Improving the feasibility pump. *Discrete Optimization* 4(1):77–86, 2007. <https://doi.org/10.1016/j.disopt.2006.10.004>
- [2] D. Baena, J. Castro. The Chebyshev center as an alternative to the analytic center in the feasibility pump. *Optimization Letters* 17(8):1757–1790, 2023. <https://doi.org/10.1007/s11590-023-02018-4>
- [3] L. Bertacco, M. Fischetti, A. Lodi. A feasibility pump heuristic for general mixed-integer problems. *Discrete Optimization* 4(1):63–76, 2007. <https://doi.org/10.1016/j.disopt.2006.10.001>
- [4] T. Berthold. Measuring the impact of primal heuristics. *Operations Research Letters* 41(6):611–614, 2013. <https://doi.org/10.1016/j.orl.2013.08.007>
- [5] T. Berthold, A. Lodi, D. Salvagnin. Ten years of feasibility pump, and counting. *EURO Journal on Computational Optimization* 7(1):1–14, 2019. <https://doi.org/10.1007/s13675-018-0109-7>

variant	with a solution		medians					completion LPs		
	inst.	runs	gap (%)	PI/TL (%)	t_{first} (s)	rounds	LP iter.	calls	feas.	infeas.
none ₋	27/248	79/1 240	100.00	100.0	19.83	354	28 622	–	–	–
none _{0.5}	23/248	70/1 240	100.00	100.0	19.75	261	32 250	–	–	–
check ₋	26/248	78/1 240	100.00	100.0	19.52	325	27 574	–	–	–
check _{0.5}	23/248	70/1 240	100.00	100.0	19.87	244	29 582	–	–	–
check _{0.75}	22/248	68/1 240	100.00	100.0	19.86	249	29 374	–	–	–
check _{0.9}	23/248	72/1 240	100.00	100.0	19.60	262	28 958	–	–	–
check _{0.95}	23/248	69/1 240	100.00	100.0	19.57	246	29 278	–	–	–
FGL ₋	26/248	78/1 240	100.00	100.0	19.82	317	27 494	3 192 751	28	3 170 429
FGL _{0.5}	23/248	69/1 240	100.00	100.0	19.82	242	29 368	3 187 843	53	3 187 787
FGL _{0.95}	23/248	69/1 240	100.00	100.0	19.73	245	28 800	3 196 657	57	3 190 506
check _{0.5} ^{fb}	11/248	27/1 240	100.00	100.0	15.40	64	44 872	–	–	–
none _{0.5} ^{fb}	9/248	24/1 240	100.00	100.0	11.11	66	47 584	–	–	–

Table 5: E2, the 248 instances of 424 on which the pilot finds nothing; columns as in Table 4. The median gap is 100 for every variant: each fails to find a solution, on its median seed, on more than half the instances.

comparison (A vs. B)	A only	B only	both	neither
<i>E1: the pilot finds a solution (176 instances)</i>				
check ₋ vs. none ₋	0	0	856	24
check _{0.5} vs. none _{0.5}	0	0	840	40
none _{0.5} vs. none ₋	8	24	832	16
check _{0.5} vs. check ₋	8	24	832	16
check _{0.95} vs. check ₋	8	24	832	16
check _{0.95} vs. check _{0.5}	0	0	840	40
FGL _{0.5} vs. check _{0.5}	0	0	840	40
FGL _{0.95} vs. check _{0.95}	0	0	840	40
FGL ₋ vs. check ₋	0	0	856	24
FGL _{0.5} vs. check ₋	8	24	832	16
check _{0.5} ^{fb} vs. check _{0.5}	6	313	527	34
none _{0.5} ^{fb} vs. none _{0.5}	6	380	460	34
<i>E2: the pilot finds nothing (248 instances)</i>				
check ₋ vs. none ₋	2	3	76	1 159
check _{0.5} vs. none _{0.5}	1	1	69	1 169
none _{0.5} vs. none ₋	17	26	53	1 144
check _{0.5} vs. check ₋	17	25	53	1 145
check _{0.95} vs. check ₋	16	25	53	1 146
check _{0.95} vs. check _{0.5}	1	2	68	1 169
FGL _{0.5} vs. check _{0.5}	0	1	69	1 170
FGL _{0.95} vs. check _{0.95}	2	2	67	1 169
FGL ₋ vs. check ₋	0	0	78	1 162
FGL _{0.5} vs. check ₋	16	25	53	1 146
check _{0.5} ^{fb} vs. check _{0.5}	12	55	15	1 158
none _{0.5} ^{fb} vs. none _{0.5}	9	55	15	1 161

Table 6: Found or not found, paired per run, for the twelve comparisons: runs on which only the first variant finds a solution, only the second, both, neither. E1 above, E2 below.

Table 7: Paired per-instance comparisons. W/L/T is the number of instances on which the first policy wins, loses, or ties against the second on the *outcome*, i.e. on the per-instance median success defined in Table 3; p is the exact two-sided sign test on the non-ties; p_{Holm} is the Holm correction over the 12 declared tests, namely 2 values of $a \times 2$ families (benchmark, non-benchmark) $\times 3$ comparisons — the *all* rows are reported for information only and are not part of the Holm family. *Faster* counts, among the instances solved by both policies, how many times the first policy’s median time to success is lower than the second’s by at least 5%, how many times the second’s is lower, and how many times the two tie within that tolerance.

Comparison	N	W/L/T	p	p_{Holm}	faster W/L/T
$a = 0.5$ — benchmark					
NAIVE vs. TEST	43	3/1/39	0.625	1.000	13/6/6
NAIVE vs. COMPLETION	43	4/4/35	1.000	1.000	12/8/4
TEST vs. COMPLETION	43	2/4/37	0.688	1.000	13/7/4
$a = 0.5$ — non-benchmark					
NAIVE vs. TEST	91	4/3/84	1.000	1.000	22/26/7
NAIVE vs. COMPLETION	91	5/3/83	0.727	1.000	33/16/5
TEST vs. COMPLETION	91	5/4/82	1.000	1.000	29/13/11
$a = 0.5$ — all					
NAIVE vs. TEST	134	7/4/123	0.549	—	35/32/13
NAIVE vs. COMPLETION	134	9/7/118	0.804	—	45/24/9
TEST vs. COMPLETION	134	7/8/119	1.000	—	42/20/15
$a = 0.9$ — benchmark					
NAIVE vs. TEST	43	1/1/41	1.000	1.000	13/10/8
NAIVE vs. COMPLETION	43	0/3/40	0.250	1.000	16/9/7
TEST vs. COMPLETION	43	0/3/40	0.250	1.000	16/6/10
$a = 0.9$ — non-benchmark					
NAIVE vs. TEST	91	3/2/86	1.000	1.000	28/25/16
NAIVE vs. COMPLETION	91	0/5/86	0.062	0.688	39/26/7
TEST vs. COMPLETION	91	0/6/85	0.031	0.375	45/7/19
$a = 0.9$ — all					
NAIVE vs. TEST	134	4/3/127	1.000	—	41/35/24
NAIVE vs. COMPLETION	134	0/8/126	0.008	—	55/35/14
TEST vs. COMPLETION	134	0/9/125	0.004	—	61/13/29

- [6] T. Berthold, A. Lodi, D. Salvagnin. *Primal Heuristics in Integer Programming*. Cambridge University Press, 2025. <https://doi.org/10.1017/9781009574792>
- [7] N. Boland, A. Eberhard, F. Engineer, A. Tsoukalas. A new approach to the feasibility pump in mixed integer programming. *SIAM Journal on Optimization* 22(3):831–861, 2012. <https://doi.org/10.1137/110823596>
- [8] N. Boland, A. Eberhard, F. Engineer, M. Fischetti, M. Savelsbergh, A. Tsoukalas. Boosting the feasibility pump. *Mathematical Programming Computation* 6(3):255–279, 2014. <https://doi.org/10.1007/s12532-014-0068-9>
- [9] S. Bolusani, M. Besançon, K. Bestuzheva, A. Chmiela, J. Dionísio, T. Donkiewicz, J. van Doornmalen, L. Eifler, M. Ghannam, A. Gleixner, et al. The SCIP Optimization Suite 9.0. ZIB Report 24-02-29, Zuse Institute Berlin, 2024. arXiv:2402.17702.
- [10] M. Cacciola, A. Forel, A. Frangioni, A. Lodi. The differentiable feasibility pump. In *Integer Programming and Combinatorial Optimization (IPCO 2025)*, Lecture Notes in Computer Science 15620, pp. 157–171, Springer, 2025. https://doi.org/10.1007/978-3-031-93112-3_12
- [11] A. Caprara, M. Fischetti, P. Toth. A heuristic method for the set covering problem. *Operations Research* 47(5):730–743, 1999. <https://doi.org/10.1287/opre.47.5.730>

- [12] S. S. Dey, A. Iroume, M. Molinaro, D. Salvagnin. Improving the randomization step in feasibility pump. *SIAM Journal on Optimization* 28(1):355–378, 2018. <https://doi.org/10.1137/16M1095962>
- [13] M. Fischetti, F. Glover, A. Lodi. The feasibility pump. *Mathematical Programming* 104(1):91–104, 2005. <https://doi.org/10.1007/s10107-004-0570-3>
- [14] M. Fischetti, A. Lodi. Local branching. *Mathematical Programming* 98(1–3):23–47, 2003. <https://doi.org/10.1007/s10107-003-0395-5>
- [15] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6(2):65–70, 1979.
- [16] M. Fischetti, D. Salvagnin. Feasibility pump 2.0. *Mathematical Programming Computation* 1(2–3):201–222, 2009. <https://doi.org/10.1007/s12532-009-0007-3>
- [17] J. Forrest et al. `coin-or/Cbc`: COIN-OR branch and cut. <https://github.com/coin-or/Cbc>; <https://doi.org/10.5281/zenodo.2720283>
- [18] A. Gleixner, G. Hendel, G. Gamrath, T. Achterberg, M. Bastubbe, T. Berthold, P. Christophel, K. Jarck, T. Koch, J. Linderoth, M. Lübbecke, H. D. Mittelmann, D. Ozyurt, T. K. Ralphs, D. Salvagnin, Y. Shinano. MIPLIB 2017: data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation* 13(3):443–490, 2021. <https://doi.org/10.1007/s12532-020-00194-3>
- [19] S. Hanafi, J. Lazić, N. Mladenović. Variable neighbourhood pump heuristic for 0–1 mixed integer programming feasibility. *Electronic Notes in Discrete Mathematics* 36:759–766, 2010. <https://doi.org/10.1016/j.endm.2010.05.096>
- [20] J. Lazić, R. Todosijejić, S. Hanafi, N. Mladenović. Variable and single neighbourhood diving for MIP feasibility. *Yugoslav Journal of Operations Research* 26(2):131–157, 2016. <https://doi.org/10.2298/YJOR140417027L>
- [21] G. Mexi, T. Berthold, D. Salvagnin. Using multiple reference vectors and objective scaling in the feasibility pump. *EURO Journal on Computational Optimization* 11:100066, 2023. <https://doi.org/10.1016/j.ejco.2023.100066>
- [22] `scipopt/scip`: Solving Constraint Integer Programs. Zuse Institute Berlin. <https://github.com/scipopt/scip>